AnchorKit: Hardware-Attested Image Commitment with Blockchain Anchoring

Jonah Owen
AnchorKit LLC, Lawrence, Kansas

Preprint — April 2026

Abstract

The integrity of photographic evidence is under increasing strain from advances in generative image synthesis. Existing provenance standards, most prominently C2PA, rely on software-level signing that can be applied post-capture to arbitrary content on any device, providing limited assurance in adversarial settings. We present AnchorKit, a hardware-attested image commitment system for Android devices. AnchorKit cryptographically binds an image hash to the device state at submission time—specifically, that the signing key resides in a hardware-backed TEE or StrongBox and was generated there, that the device booted from a verified OS image, and that submission occurred within a five-minute nonce window—and anchors the resulting commitment to the Solana blockchain. The system does not claim to prove that the image hash corresponds to a specific physical scene (the camera pipeline trust boundary is an explicit residual risk); it proves that the hash was submitted under the stated hardware and protocol constraints. Daily batches of image hashes are aggregated into Merkle trees whose roots are posted to a custom Solana programme, enabling individual inclusion proofs verifiable offline using only SHA-256 and a public Solana RPC endpoint. We describe the system architecture, formal system model, cryptographic constructions, threat analysis, and a formal comparison with C2PA. We identify the camera pipeline trust boundary as a residual architectural risk distinct from the analogue hole; proposed engineering mitigations are discussed separately in Appendix **??**.

Keywords. hardware attestation, image commitment, blockchain anchoring, Android Keystore, Merkle trees, C2PA, verified boot, trusted execution environment.

ACM CCS 2012. *Security and privacy* → Hardware-based security protocols; *Security and privacy* → Digital signatures; *Security and privacy* → Public key (asymmetric) cryptographic techniques.

1 Introduction

The provenance of digital photographs has become a matter of significant practical consequence. Photographic evidence is routinely submitted in legal proceedings, cited in journalism, and used to inform public discourse. The emergence of high-fidelity generative models capable of producing synthetic images visually indistinguishable from genuine photographs creates an urgent need for mechanisms that can attest not merely that an image file carries a digital signature, but that the image was captured on genuine hardware, on an unmodified device, at a specific point in time, with no opportunity for post-capture substitution.

The C2PA (Coalition for Content Provenance and Authenticity) standard, supported by Adobe, Microsoft, Intel, Arm, BBC and others, provides a framework for embedding signed manifests into media files. C2PA represents a meaningful advance over unsigned media, but its trust model has structural limitations: credentials can be issued to software applications and

used to sign arbitrary content at any time, manifest stripping is routine in publishing pipelines, and verification depends on centralised services.

AnchorKit enforces a set of joint constraints—hardware key generation, verified boot state, time-bounded submission, and decentralised anchoring—that together prevent delayed attestation (signing outside a bounded nonce window) and key substitution (use of a software-generated or imported key) that are structurally possible under C2PA, within the trust model defined in Section **??**. The design does not introduce new cryptographic primitives; its contribution is in enforcing these constraints simultaneously and verifiably, under explicit assumptions, in a deployed system. The four mechanisms are:

1. **Android Key Attestation** — cryptographic proof that the signing key resides in a TEE or StrongBox and was never exported
2. **Time-bounded nonce binding** — a 5-minute expiry window that makes retroactive attestation infeasible
3. **Merkle tree commitment** — efficient batching of daily submissions with compact individual inclusion proofs
4. **Solana blockchain anchoring** — append-only, decentralised, offline-verifiable commitment of daily Merkle roots

Prior systems employing blockchain anchoring (e.g., Numbers Protocol [**?**]) do not enforce hardware attestation or verified boot state. C2PA-capable camera hardware performs hardware signing but does not anchor to a decentralised ledger and permits post-capture manifest application in software. In surveying related systems, we have not identified a deployed system simultaneously enforcing all four constraints; we make no stronger novelty claim without a complete literature survey.

Contributions. This paper makes the following contributions:

- A deployed hardware-attested image commitment system that enforces hardware key generation, verified boot state, and time-bounded submission simultaneously, preventing—within the trust boundaries of Section **??**—delayed attestation and key substitution attacks that are structurally permitted by C2PA
- A nightly Merkle batching architecture reducing per-record blockchain cost to near zero while preserving individual inclusion proofs verifiable without contacting any AnchorKit-operated service
- A custom Solana programme storing Merkle roots in a chunked linked-list PDA structure with fully offline verification
- A security analysis separating cryptographic guarantees (message binding under ECDSA-UF-CMA; Merkle integrity under SHA256-CR) from system trust assumptions and protocol design properties, with explicit acknowledgement of the boundaries between them
- Identification of the camera pipeline trust boundary as a residual architectural risk distinct from the analogue hole, with a formal oracle model delineating what is and is not within scope of the cryptographic proofs

2 Background

2.1 Android Hardware Attestation

Android's Keystore system provides applications with access to cryptographic keys stored in hardware-backed environments—either a Trusted Execution Environment (TEE) or a dedicated StrongBox security chip. Keys generated in these environments cannot be extracted: private key material never appears in normal processor memory and is inaccessible even to the device OS.

Android Key Attestation, available from Android 7.0, extends this with a certification mechanism. When a key is generated in hardware, the Keystore produces an X.509 certificate chain signed by a Google-operated attestation CA. A custom extension (OID `1.3.6.1.4.1.11129.2.1.17`) carries structured metadata including the key's security level (`Software=0`, `TrustedEnvironment=1`, `StrongBox=2`), its origin (`GENERATED=0`, `IMPORTED=2`), and the device's verified boot state (`Verified=0`, `SelfSigned=1`, `Unverified=2`, `Failed=3`). Prior work has used Android Key Attestation for mobile payments and device authentication; its application to image commitment combining all four of the properties described above has not, to our knowledge, been previously published. We survey related systems in Section **??**.

2.2 Merkle Trees and Tamper-Evident Logs

Merkle hash trees [**?**] commit to an ordered set of values in a manner permitting compact membership proofs of size $O(\log n)$. Certificate Transparency [**?**] deployed this construction at scale for TLS certificates, posting Merkle roots to append-only logs. AnchorKit applies the same construction to image hash records, posting daily roots to Solana. A CT-style transparency log would provide equivalent Merkle integrity guarantees; AnchorKit uses Solana for two reasons beyond cryptographic integrity. First, CT log operators can equivocate—presenting different views of the log to different parties—whereas Solana programme account state is globally consistent under consensus. Second, Solana is readable by any party via a public RPC endpoint without permission from any AnchorKit-operated service, whereas a CT log requires the log operator to remain available. These properties come at the cost of Solana's probabilistic rather than absolute immutability (Section **??**).

2.3 Content Provenance Standards

C2PA [**?**] defines a manifest format for embedding provenance assertions into media files, signed using X.509 credentials from C2PA-accredited CAs. Credentials can be issued to software applications, permitting signing of arbitrary content at any time. Manifests are silently stripped by many image processing tools and social media platforms. Verification requires contacting centralised services. We compare AnchorKit to C2PA formally in Section **??**.

3 Related Work

Blockchain-based media provenance. Numbers Protocol [**?**] posts image hashes to Ethereum-compatible networks, providing timestamped records of image registration. Like AnchorKit, it uses hash-based anchoring rather than manifest embedding. Unlike AnchorKit, it does not enforce hardware-backed key generation or verified boot state: any device, including one using a software key, can register a hash. The per-transaction cost model does not scale efficiently to high submission volumes; AnchorKit's batched Merkle approach reduces per-record on-chain cost to near zero.

Guardian Project ProofMode. ProofMode [**?**] captures sensor context—GPS coordinates, accelerometer readings, device identifiers—alongside images and cryptographically signs the

bundle at capture time. ProofMode establishes rich environmental context that AnchorKit's hash commitment does not capture. Conversely, ProofMode does not enforce hardware-backed key generation, verified boot state, or nonce-bounded submission windows, and does not anchor commitments to a decentralised ledger. The two systems are complementary rather than competing.

Starling Framework. The Starling Capture framework [**?**] applies C2PA-based signing to image capture for documentary and legal evidence contexts. As a C2PA implementation it inherits the trust model analysed in Section **??**: credentials can be issued to software applications and applied post-capture. Starling has produced significant work on deployment workflows and institutional adoption; AnchorKit addresses complementary gaps at the cryptographic and hardware trust levels.

C2PA hardware implementations. Leica and Sony have shipped cameras with hardware-based C2PA signing, where the signing key resides in a dedicated security chip on the camera body, enforcing hardware key generation analogously to AnchorKit's TEE requirement. These implementations sign C2PA manifests rather than computing blockchain-anchored Merkle commitments; manifest stripping remains a concern in downstream publishing pipelines, and verification requires contacting centralised C2PA verification services.

Android Key Attestation in prior systems. Android Key Attestation has been applied to mobile payments and device authentication [**?**], where the ability to prove that a key resides in hardware is used to assert device legitimacy to a remote service. To our knowledge, no prior published system applies Android Key Attestation to image commitment in combination with time-bounded nonce binding, Merkle batching, and decentralised blockchain anchoring. We make no stronger novelty claim without a complete systematic literature review.

4 System Architecture

4.1 Overview

AnchorKit comprises three principal components: an Android SDK running on the capture device; a Python/FastAPI backend that validates attestations and manages the submission pipeline; and a Rust Solana programme storing Merkle roots on-chain. A Rust Merkle tree builder handles nightly batch processing as a performance-critical subprocess of the backend.

4.2 Android SDK

Key generation. On first use, the SDK generates an EC P-256 key pair inside the Android Keystore with `PURPOSE_SIGN`, `STRONGBOX_BACKED` requested where available with fallback to the TEE. The private key material is never accessible to application code or any other process on the device.

Capture and hashing. The SDK captures images via CameraX and computes SHA-256 of the captured frame in memory before any write to the device filesystem, ensuring the hash corresponds to the raw captured image rather than a subsequent on-disk encoding.

Attestation and signing. Prior to submission, the SDK fetches a single-use 256-bit nonce from the server with a five-minute TTL. The device constructs the signed message:

$$m = \texttt{hash} \parallel \texttt{:} \parallel \texttt{nonce} \parallel \texttt{:} \parallel \texttt{metadataHash}$$

where `metadataHash` is the SHA-256 of canonicalised metadata (key-value pairs sorted lexicographically, joined with commas). This message is signed via ECDSA-SHA-256 using the hardware-backed key, binding metadata to the attested payload and preventing in-transit modification of timestamp or dimension fields. The Keystore simultaneously produces an attestation certificate chain for the signing key.

Client-side defences. The SDK inspects `Build.TAGS` for `test-keys`, scans for root management artefacts, and implements programmatic SPKI SHA-256 certificate pinning on `HttpsURLConnection`, bypassing any device-level `network_security_config.xml` overrides.

4.3 Backend Validation Pipeline

The submission endpoint performs the following validation steps in sequence:

1. **Hash format** — strict regex requiring exactly 64 lowercase hexadecimal characters
2. **API key validation**
3. **Atomic nonce consumption** — a DynamoDB conditional update sets `used=True` only if `used=False` and the nonce has not expired; concurrent requests with the same nonce result in exactly one success
4. **Certificate chain validation** — each certificate's temporal validity is checked; the chain is cryptographically verified; the root fingerprint must appear in the operator-configured set of Google attestation CA certificates
5. **Enclave signature verification** — the leaf certificate's public key verifies the ECDSA signature over m reconstructed server-side from the submitted metadata
6. **Attestation extension enforcement** — three fields are enforced with explicit absence rejection:
   - `attestationSecurityLevel` $\in \{1, 2\}$ (TEE or StrongBox; `Software=0` rejected)
   - `hardwareEnforced.origin = GENERATED` (0); `IMPORTED`, `DERIVED`, and absent rejected
   - `hardwareEnforced.rootOfTrust.verifiedBootState = Verified` (0); all other values and absent rejected

Absence rejection—treating a missing field identically to an invalid value—defends against omission-bypass attacks in which a crafted certificate simply omits a required field.

4.4 Storage, Batching, and Blockchain Anchoring

Submissions are written to DynamoDB in a three-table rotation (A/B/C) with tables swapping at midnight UTC. Each record receives a sequential `hash_id` via atomic counter, establishing deterministic Merkle leaf ordering.

At midnight UTC, the nightly batch filters records by `day == yesterday`, sorts by `hash_id`, writes a compressed JSONL archive to S3 with a sharded lookup index (enabling $O(1)$ hash-to-day lookup), constructs a Merkle tree over the ordered hash sequence using a Rust implementation, and posts the resulting root to the Solana programme via the `add_merkle_root` instruction.

The Solana programme stores roots in programme-derived accounts organised as a chunked linked list. Each entry contains the 32-byte Merkle root, a UTF-8 date string, and a Unix timestamp. The borsh serialisation is deterministic and language-independent, allowing verification using any borsh library.

Merkle proof verification. Let h be the image hash and $\pi = [(s_1, d_1), \ldots, (s_k, d_k)]$ the inclusion proof. The verifier computes $v_0 = h$ and iterates:

$$v_j = \begin{cases} \text{SHA-256}(s_j \parallel v_{j-1}) & d_j = \text{left} \\ \text{SHA-256}(v_{j-1} \parallel s_j) & d_j = \text{right} \end{cases}$$

checking v_k against the on-chain root retrieved directly from the Solana RPC. This requires no contact with any AnchorKit-operated infrastructure.

4.5 Formal System Model

We define a formal abstraction of the AnchorKit pipeline that delineates the trust boundary between the camera pipeline (explicitly untrusted) and the cryptographic pipeline (TEE-backed). Security properties in Section **??** are stated with respect to this model.

Capture oracle $\mathcal{C}$. On input a scene $\mathcal{S}$, returns a pair (h, meta) where $h \in \{0,1\}^{256}$ is a hash value and meta is a structured metadata record. The oracle models the Android camera pipeline from sensor to userspace: frame data traverses the camera HAL, Android camera service, and CameraX API before being hashed. $\mathcal{C}$ is *not trusted*: an adversary who controls the camera service may influence h independently of the physical scene, without disturbing the verified boot state on certain device configurations.

Attestation oracle $\mathcal{AT}$. On input device state δ, returns an X.509 certificate chain cert whose attestation extension reflects the security level, key origin, and boot state of the hardware-backed key on device δ. $\mathcal{AT}$ is trusted: we assume Google's attestation CA faithfully reflects device hardware properties (Section **??**). A compromised attestation CA would corrupt this oracle's output.

Signing oracle $\mathcal{S}_{\mathsf{sign}}$. On input a message m, invokes the Android Keystore on behalf of the hardware-backed key sk, returning an ECDSA signature σ. The private key material is never exposed to the caller. The adversary cannot query $\mathcal{S}_{\mathsf{sign}}$ for a sk it does not hold.

Nonce oracle $\mathcal{N}$. On first query, returns a fresh 256-bit nonce ν and records $\mathsf{used}(\nu) = \bot$ with TTL of 5 minutes. On subsequent queries with the same ν: if $\mathsf{used}(\nu) = \top$ or ν has expired, returns $\bot$; otherwise sets $\mathsf{used}(\nu) = \top$ and returns $\top$. $\mathcal{N}$ models DynamoDB conditional update atomicity and is trusted to enforce single-use semantics.

Commitment oracle $\mathcal{CM}$. On input a Merkle root R, appends R to an append-only sequence of committed roots. $\mathcal{CM}$ models Solana blockchain anchoring and is trusted under the Solana consensus assumption (Section **??**) to be append-only and globally consistent.

Valid submission. A valid AnchorKit submission proceeds as follows:

1. The device queries $\mathcal{N}$ to obtain nonce ν.
2. The device queries $\mathcal{C}$ to obtain (h, meta).
3. The device forms $m = h \parallel : \parallel \nu \parallel : \parallel \text{SHA-256}(\mathsf{meta})$ and queries $\mathcal{S}_{\mathsf{sign}}(m)$ to obtain σ.
4. The device queries $\mathcal{AT}(\delta)$ to obtain cert.
5. The backend validates $(h, \nu, \mathsf{meta}, \sigma, \mathsf{cert})$: verifies nonce freshness via $\mathcal{N}$, verifies σ against cert's public key, and enforces attestation field constraints.

6. At midnight UTC, the backend forms a Merkle tree over all accepted h values and queries $\mathcal{CM}(R)$.

Scope of formal claims. Theorem **??** concerns the unforgeability of step 3 under $\mathcal{S}_{\mathsf{sign}}$. Theorem **??** concerns the binding of accepted h values through $\mathcal{CM}$. Neither theorem addresses the output of $\mathcal{C}$: whether h faithfully represents the physical scene is outside the formal model and is identified as a residual risk in Section **??**.

5 Security Analysis

5.1 Adversary Model

We consider a computationally bounded (PPT) adversary who: controls the network between the capture device and the backend server; may observe all submitted payloads and server responses; may hold valid API keys and operate genuine Android devices of their own; and may attempt to cause the backend to accept a submission whose hash does not correspond to an image captured on a genuine, unmodified device at the time of submission.

The adversary may not break ECDSA-UF-CMA or SHA256-CR. The adversary does not control Google's attestation CA infrastructure, Solana's consensus mechanism, or DynamoDB's conditional update semantics; these are treated as trust boundaries described in Section **??**, not as cryptographic assumptions. The adversary cannot export private key material from a hardware-backed TEE or StrongBox, and cannot modify Android's verified boot state without causing the boot chain verification to fail.

AnchorKit's security properties span three distinct layers that require different analytical treatments. We address each separately rather than blending them, which would obscure which claims rest on cryptographic hardness and which rest on engineering design or operational trust.

5.1.1 Unified Security Goal

We state precisely what AnchorKit is and is not designed to prove.

What is proven end-to-end (Section ??). Given the adversary model above and Assumptions **??**–**??**:

1. *Message binding* — no PPT adversary can produce a valid ECDSA signature over a fresh message without access to the hardware-backed private key (Theorem **??**). Any accepted submission therefore carries a signature that could only have been produced by the TEE holding that key.

2. *Merkle inclusion integrity* — no PPT adversary can produce a valid inclusion proof for an image hash not present in the committed set (Theorem **??**). Any verifying inclusion proof therefore certifies that the specific hash was included in the batch whose root is on-chain.

What is asserted under trust assumptions, not proven (Section ??). The following hold only if the named external parties behave correctly:

- *Hardware faithfulness* — that the attested key is backed by genuine hardware depends on Google's attestation CA operating correctly. A compromised CA could produce attestations for software keys.

- *Blockchain immutability* — that on-chain roots are not retroactively modified depends on Solana's consensus mechanism not being successfully attacked.

- *Nonce uniqueness* — that a nonce cannot be reused depends on DynamoDB's conditional update semantics functioning as specified.

What is explicitly outside scope.

- AnchorKit does not prove that the signed image hash corresponds to output from the camera sensor. The camera pipeline trust boundary (Section **??**) is a residual architectural gap: on a verified-boot device, frame data traverses userspace before hashing, and an adversary who compromises the camera service without disturbing boot state can substitute frame buffers. The cryptographic proofs of Theorems **??** and **??** do not address this boundary.
- AnchorKit does not prove anything about the content of the scene in front of the camera (the analogue hole).
- AnchorKit does not prove that the capture time embedded in metadata is accurate; only that the metadata hash was included in the signed message.

5.2 Cryptographic Layer

Two properties of AnchorKit admit standard cryptographic proofs by reduction to well-established hardness assumptions.

Assumption 1 (SHA256-CR)**.** *SHA-256 is collision resistant: for all* PPT *adversaries* $\mathcal{A}$*,*

$$\Pr\Big[x \neq y \wedge \textit{SHA-256}(x) = \textit{SHA-256}(y) \ : \ (x, y) \leftarrow \mathcal{A}(1^\lambda)\Big] \leq \mathsf{negl}(\lambda).$$

Assumption 2 (ECDSA-UF-CMA)**.** *ECDSA is existentially unforgeable under chosen message attack: for all* PPT *adversaries* $\mathcal{A}$ *with access to a signing oracle* $\mathcal{O}_{\text{sign}}$*,*

$$\Pr\Big[\mathsf{Verify}(pk, m^*, \sigma^*) = 1 \wedge m^* \notin Q \ : \ (pk, sk) \leftarrow \mathsf{Gen}(1^\lambda);\ (m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{sign}}(sk,\cdot)}(pk)\Big] \leq \mathsf{negl}(\lambda),$$

where Q *is the set of messages submitted to* $\mathcal{O}_{\text{sign}}$*.*

Definition 1 (Message Binding)**.** *The game* $\mathsf{BIND}_{\mathcal{A}}(\lambda)$*:*

- *A hardware-backed key pair* (pk, sk) *is generated inside a TEE;* $\mathcal{A}$ *receives* pk *and may query* $\mathcal{O}_{\text{sign}}(sk, \cdot)$*.*
- $\mathcal{A}$ *wins if it outputs* (m^*, σ^*) *with* $m^* \notin Q$ *and* $\mathsf{Verify}(pk, m^*, \sigma^*) = 1$*.*

Definition 2 (Merkle Inclusion Integrity)**.** *Let* $S = \{h_1, \ldots, h_n\}$ *be a set of image hashes committed to Merkle root* R*. The game* $\mathsf{MERKLE}_{\mathcal{A}}(\lambda)$*:*

- $\mathcal{A}$ *receives* R *and the Merkle tree over* S*.*
- $\mathcal{A}$ *wins if it outputs* $h^* \notin S$ *and proof* π^* *such that Merkle verification accepts* (h^*, π^*, R)*.*

Theorem 1 (Message Binding)**.** *Under* ECDSA-UF-CMA*,* $\Pr[\mathsf{BIND}_{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$ *for all* PPT $\mathcal{A}$*.*

Proof. Suppose $\mathcal{A}$ wins $\mathsf{BIND}_{\mathcal{A}}(\lambda)$ with non-negligible probability ϵ. We construct a PPT reduction $\mathcal{B}$ that breaks ECDSA-UF-CMA with probability ϵ. $\mathcal{B}$ receives challenge public key pk^* from the ECDSA-UF-CMA challenger, runs $\mathcal{A}(pk^*)$, forwards all signing queries to its own oracle, and outputs $\mathcal{A}$'s forgery (m^*, σ^*) directly. Since $m^* \notin Q$ and $\mathsf{Verify}(pk^*, m^*, \sigma^*) = 1$ with probability ϵ, $\mathcal{B}$ breaks ECDSA-UF-CMA with probability ϵ. By Assumption **??**, $\epsilon \leq \mathsf{negl}(\lambda)$. □

Remark 1. *The signed message* $m = \texttt{hash} \| : \| \texttt{nonce} \| : \| \texttt{metadataHash}$ *binds the image hash, nonce, and SHA-256 of all metadata fields simultaneously. Any in-transit modification of the hash, nonce, timestamp, or GPS coordinates produces a distinct* m*, invalidating the signature. Resistance to such modification reduces directly to* $\mathsf{BIND}_{\mathcal{A}}(\lambda)$ *and hence to ECDSA-UF-CMA.*

Theorem 2 (Merkle Inclusion Integrity)**.** *Under* SHA256-CR*,* $\Pr[\mathsf{MERKLE}_{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$ *for all* PPT $\mathcal{A}$*.*

Proof. Suppose $\mathcal{A}$ produces $h^* \notin S$ and proof $\pi^* = [(s_1, d_1), \ldots, (s_k, d_k)]$ such that Merkle verification accepts (h^*, π^*, R) with non-negligible probability ϵ.

Verification computes $v_0 = h^*$ and for $j = 1, \ldots, k$:

$$v_j \;=\; \begin{cases} \text{SHA-256}(s_j \,\|\, v_{j-1}) & d_j = \text{left} \\ \text{SHA-256}(v_{j-1} \,\|\, s_j) & d_j = \text{right} \end{cases}$$

with $v_k = R$. Let i^* denote the leaf position implied by the direction sequence $(d_1, \ldots, d_k)$, and let $t_1, \ldots, t_k$ be the *honest* sibling nodes along the path from leaf i^* to the root in the committed Merkle tree over S. Define the honest path $u_0 = h_{i^*}$ and u_j by the same recurrence using honest siblings t_j:

$$u_j \;=\; \begin{cases} \text{SHA-256}(t_j \,\|\, u_{j-1}) & d_j = \text{left} \\ \text{SHA-256}(u_{j-1} \,\|\, t_j) & d_j = \text{right} \end{cases}$$

By construction of the Merkle tree over S, honest recomputation along the path to leaf i^* yields $u_k = R$: each u_j is the actual internal node at depth j on this path, and u_k is the root committed to R.

We have $v_k = u_k = R$ and $v_0 = h^* \neq h_{i^*} = u_0$ (since $h^* \notin S$). Therefore there exists a smallest index $\ell \in \{1, \ldots, k\}$ such that $v_\ell = u_\ell$ but $v_{\ell-1} \neq u_{\ell-1}$. At level ℓ, the adversary computes $v_\ell = \text{SHA-256}(\text{combine}(s_\ell, v_{\ell-1}))$ and the honest path gives $u_\ell = \text{SHA-256}(\text{combine}(t_\ell, u_{\ell-1}))$, with $v_\ell = u_\ell$. Since $v_{\ell-1} \neq u_{\ell-1}$, the two SHA-256 inputs are distinct regardless of whether $s_\ell = t_\ell$ (concatenation is injective in each argument). This is a SHA-256 collision. A PPT reduction $\mathcal{B}$ that runs $\mathcal{A}$, locates ℓ, and outputs the two colliding preimages breaks SHA256-CR with probability ϵ. By Assumption **??**, $\epsilon \leq \mathsf{negl}(\lambda)$. □

5.3 System Trust Model

Several of AnchorKit's security properties depend on the correct operation of external infrastructure. These are trust assumptions, not cryptographic hardness assumptions; they cannot be reduced to computational problems and should not be presented as such.

Hardware attestation trust. The security of the attestation chain rests on the assumption that Google's Android attestation CA private key is not compromised, and that the X.509 extension values in Google-issued certificates faithfully reflect device hardware properties. More precisely, attestation proves device state *as claimed by the attestation extension*; the mapping from those field values to actual hardware behaviour depends on the correctness of Google's CA issuance policy and the Android platform implementation. A compromise of Google's attestation CA private key would allow fabrication of attestation chains claiming hardware-backing for software keys. Certificate Transparency applied to attestation certificates would provide an auditable check on this assumption; this is identified as future work in Section **??**.

Blockchain immutability. AnchorKit relies on the assumption that committed Solana programme account data is not retroactively altered. This holds under Solana's consensus protocol and economic security model, except with probability bounded by the probability of a successful consensus attack or chain reorganisation. Blockchains are not mathematically immutable;

their immutability is probabilistic and economically grounded. The formal analysis of Solana's consensus security is outside the scope of this paper; we cite the Solana whitepaper [**?**] for the underlying claims.

Nonce uniqueness. The nonce protocol relies on DynamoDB's conditional update atomicity to prevent double-use. This is an engineering property of a distributed database system, not a cryptographic primitive. Formal verification of the nonce protocol using tools such as ProVerif, Tamarin, or TLA$^+$ is identified as future work. We note that nonce compromise does not threaten the cryptographic binding properties proven in Section **??**: a reused nonce permits a replay of an existing valid submission but does not enable forging a signature for a new image hash.

5.4 Protocol Design Properties

The following properties hold by construction of the validator logic. They are correctness claims about the implementation design, not cryptographic theorems.

Root fingerprint enforcement. The certificate chain validator rejects any chain whose root certificate fingerprint does not appear in the operator-configured Google attestation CA set. A chain rooted at a self-signed or third-party certificate is rejected unconditionally by this check, regardless of the contents of the attestation extension.

Omission-bypass prevention. The attestation extension parser explicitly rejects submissions in which the `origin` or `verifiedBootState` fields are absent, treating absence identically to an invalid value. This design choice prevents an attack in which a crafted certificate omits required extension fields to avoid failing individual field checks while still passing the overall extension validation.

Attestation field enforcement. The combined effect of root fingerprint enforcement, omission-bypass prevention, and explicit field-value checks (`attestationSecurityLevel` $\in \{1, 2\}$; `origin = GENERATED`; `verifiedBootState = Verified`) means that any accepted submission's certificate chain must have been issued by Google's CA infrastructure attesting to the stated hardware properties—subject to the hardware attestation trust assumption in Section **??**.

5.5 Residual Risks

Google Attestation CA compromise. A compromise of Google's attestation CA private key would allow fabrication of certificate chains accepted by AnchorKit's validator, undermining the hardware attestation trust assumption. Past blockchain records remain intact under the blockchain immutability assumption; the compromise would affect only future submissions.

Solana programme authority key compromise. Compromise of the authority key permits posting of false Merkle roots for future dates only. Under Solana's consensus assumption, existing on-chain records remain intact, and the compromise is detectable by any party monitoring the programme account.

The analogue hole. AnchorKit attests to the integrity of the capture pipeline and device. It makes no claim about the content of the scene in front of the camera. A device pointed at a screen displaying synthetic content produces a valid attestation. This is a fundamental limitation of any camera-based commitment system.

Camera pipeline trust boundary. Theorems **??** and **??** concern the cryptographic binding of the signed message. They do not cover the data pipeline between the camera sensor and the point at which the hash is computed. On a verified-boot device, image data traverses the camera HAL, Android camera service, and CameraX API in normal userspace before reaching the hashing routine. An adversary able to compromise the camera service without disturbing the verified boot state could substitute frame buffers prior to hashing. This attack is substantially constrained by Android SELinux policy on verified-boot devices, but represents a residual architectural gap not addressed by the cryptographic proofs above. Engineering mitigations are discussed in Appendix **??**.

AWS infrastructure dependency. The submission pipeline depends on DynamoDB and S3. An outage prevents new submissions but does not affect records already anchored to Solana.

6 Comparison with C2PA

Table 1: AnchorKit vs. C2PA: structural property comparison.

Property	**C2PA**	**AnchorKit**
Signing environment	Any software or HSM with a C2PA credential	Hardware TEE or StrongBox on the capture device
Post-capture signing	Permitted	Prevented under nonce trust model (Sec. **??**) — nonce expires in 5 minutes
Device integrity verified	Not enforced	`verifiedBootState = Verified` enforced
Hardware key generation enforced	Not enforced	`origin = GENERATED` enforced
Credential stripping	Manifests stripped by many tools and platforms	Hash-based; no embedded credential to strip
Verification infrastructure	Centralised CA + web verification service	Solana blockchain; direct RPC query
Offline verification	Requires centralised service	SHA-256 computation + Solana RPC only
Impact of key compromise	Retroactive forgery of past-dated records possible	Past records immutable; only future records affected

The most significant structural difference concerns credential abuse. A C2PA signing credential, once issued to a software application, can sign any image at any time; there is no mechanism preventing a credential holder from producing a signed manifest for synthetically generated content or for an image captured on a different device. AnchorKit's five-minute nonce TTL prevents retroactive attestation under the nonce and attestation trust assumptions of Section **??**.

C2PA's manifest-embedding approach co-locates provenance data with the image file, simplifying some distribution workflows. AnchorKit's hash-based approach means the provenance record survives image processing, compression, and format conversion that silently strip C2PA manifests.

7 Limitations and Future Work

Platform scope. AnchorKit is Android-only. Apple does not expose the equivalent of Android Key Attestation for third-party verification of key hardware-binding and device integrity state. iOS support via Apple's App Attest API would provide a meaningfully weaker but potentially useful provenance signal for that platform.

Time commitment granularity. The blockchain commitment establishes that a hash was submitted before midnight UTC on a given day. Integrating RFC 3161 trusted timestamping [**?**] into the submission payload would provide a cryptographically verifiable capture time with sub-second granularity, independently auditable against the TSA's own logs.

Camera pipeline mitigations. Two engineering approaches to narrowing the camera pipeline trust boundary—IMU sensor fusion binding and a hardware-isolated ISP-to-TEE pipeline—are described in Appendix **??**. Neither constitutes a cryptographic guarantee within the formal model of Section **??**; both are discussed there as engineering controls and future research directions rather than as security properties of the current system.

Multi-chain anchoring. Posting Merkle roots to multiple independent blockchains would reduce dependence on Solana's continued operation and provide defence against chain-level governance failures.

Zero-knowledge content proofs. For use cases requiring proof that an image depicts a particular subject without revealing the image itself, integration with zero-knowledge proof schemes represents a promising direction for future work.

8 Conclusion

AnchorKit addresses a well-defined gap in the current landscape of image commitment systems: the absence of a deployed system providing cryptographic proof that an image hash was submitted from a device attesting to hardware-backed key generation and verified OS boot state, with a time-bounded nonce preventing delayed submission, and that the resulting commitment has been anchored to a decentralised, globally consistent ledger accessible to any verifier without contacting any proprietary service.

The system achieves this by composing Android Key Attestation, time-bounded nonce binding, Merkle tree commitment, and Solana blockchain anchoring with a rigorous validation pipeline that enforces all required attestation properties and rejects both invalid values and absent fields. The result provides stronger guarantees than C2PA in specific adversarial settings—particularly with respect to delayed attestation, hardware key generation enforcement, device integrity verification, and immutability of past records under key compromise—within the trust boundaries defined explicitly in Section **??**.

We acknowledge the residual risks that cannot be addressed within the current design: the Google attestation CA trust dependency, the analogue hole, and the camera pipeline trust boundary. The last of these motivates the most significant direction for future work: a hardware-isolated camera pipeline routing frame data from the ISP to the TEE before any userspace exposure, eliminating the remaining manipulable boundary in the capture chain.

AnchorKit does not claim to solve the general problem of image authenticity. It provides a cryptographically sound, hardware-attested, blockchain-anchored proof of a specific and well-defined set of properties: that a particular image hash was submitted under named hardware and protocol constraints, as specified in the formal model of Section **??**. Whether that hash faithfully represents a physical scene is outside the system's formal scope. For many real-world

commitment questions—establishing that a specific hash existed at a specific time on a device satisfying specific hardware constraints—that bounded claim is exactly what is needed.

References

[1] R. C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, LNCS 293, pp. 369–378. Springer, 1987.

[2] B. Laurie, A. Langley, and E. Kasper. Certificate Transparency. RFC 6962, IETF, 2013.

[3] Coalition for Content Provenance and Authenticity. *C2PA Technical Specification*, Version 1.3, 2022.

[4] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato. Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). RFC 3161, IETF, 2001.

[5] Android Developers. Verifying hardware-backed key pairs with Key Attestation, 2023.

[6] A. Yakovenko. Solana: A new architecture for a high performance blockchain. Whitepaper v0.8.13, 2018.

[7] D. Cooper et al. Internet X.509 Public Key Infrastructure Certificate and CRL Profile. RFC 5280, IETF, 2008.

[8] Numbers Protocol. Numbers Protocol: Decentralised Photo Network, 2022.

[9] Guardian Project. ProofMode: Mobile media authentication with sensor data. Technical report, Guardian Project, 2017. Available at `https://proofmode.org`.

[10] Starling Lab. The Starling Framework: Integrity verification for digital media. Technical report, Starling Lab (USC Shoah Foundation and Stanford University), 2021. Available at `https://starlinglab.org`.

A Engineering Mitigations for the Camera Pipeline Trust Boundary

The following proposals address the camera pipeline trust boundary identified in Section **??**. They are engineering controls, not cryptographic guarantees. Neither falls within the formal model of Section **??**, and neither provides a provable security bound reducible to Assumptions **??** or **??**. They are included here rather than in the main body because the paper's formal claims do not depend on them.

A.1 IMU Sensor Fusion Binding

Binding inertial measurement unit (IMU) data—accelerometer and gyroscope readings sampled within a defined window around capture—into the signed payload is proposed as a complementary engineering control. The signed message would be extended to:

$$m = \texttt{hash} \parallel : \parallel \texttt{nonce} \parallel : \parallel \texttt{metadataHash} \parallel : \parallel \texttt{imuHash}$$

where `imuHash` is the SHA-256 of a structured record of timestamped accelerometer and gyroscope readings captured in the 500 ms window surrounding the capture event.

Limitations. This is a heuristic without a provable security bound. An adversary with OS-level access sufficient to inject synthetic frames into the camera pipeline can also supply fabricated IMU readings drawn from any distribution, including one calibrated to resemble genuine handheld motion. Furthermore, the statistical distinguishability of genuine handheld capture motion from adversarially synthesised profiles has not been established in the published literature; we are not aware of peer-reviewed work formally characterising this boundary. IMU binding should be understood as raising the engineering cost of a camera pipeline attack, not as a cryptographic closure of the trust boundary. Empirical evaluation of its robustness under adversarial conditions is identified as future work.

A.2 Hardware-Isolated Camera Pipeline

The camera pipeline trust boundary is most completely addressed by routing image data from the camera ISP directly to the TEE for hashing before any userspace exposure. In this model the TEE signs the frame hash, capture timestamp, and sensor configuration registers; frame data never appears in normal processor memory. The capture oracle $\mathcal{C}$ of Section **??** would become trusted in this architecture.

Current status. This construction is not achievable via current public Android APIs, which do not expose an ISP-to-TEE path for third-party applications. Realising it requires OEM partnership to modify the camera HAL and TEE firmware, or collaboration with a silicon vendor. A hardware reference design and TEE application implementing ISP-to-TEE frame hashing would represent a commitment system whose formal model closes the camera pipeline trust boundary entirely, eliminating the $\mathcal{C}$-is-untrusted caveat in Section **??**. We identify this as the most significant open engineering direction.